

arXiv template vote

Four industry templates (A to D) and three title styles of our own class (E to G). Each candidate gets one sheet with its first two pages at full size.

	Candidate	Source	What it does well	What we left behind
A	ByteDance Seed	arXiv 2602.10885	Logo in the page-one header, title centred between two thin brand rules, outlined abstract panel with date and links inside	Every heading in brand blue, body bold switched to sans, a proprietary TTF font patched in with pdfmapline
B	Meta FAIR	arXiv 2412.06769	The whole front matter in one tinted card, the \author[1] and \affiliation[1] API, metadata lines, author-year citations	Logo drawn as an overlay that long metadata can run into, a proprietary font
C	ServiceNow (our Mem- π)	arXiv 2605.21463	The Meta card on white with a brand-colour frame, GitHub and Hugging Face link lines with icons	A heavy 2 pt saturated frame, asset paths hard-coded to one project folder
D	LeapLab, Tsinghua	arXiv 2504.13837	Logo left and date right in the page-one header, abstract on a soft grey panel, project page line	An 840-line fork of the ICML style whose branding stops at the logo
E	Ours, titlestyle=card	udemmila.cls	B and C combined, with a purple-to-blue edge and the UdeM and Mila logos set beside the metadata instead of over it	
F	Ours, titlestyle=band	udemmila.cls	New, a full-width purple band with white logos across the top of page one	
G	Ours, titlestyle=rule	udemmila.cls	A and D combined, colour logos in the header, ruled centred title, out-lined abstract	

All three of ours share everything below the title block. Inter (the closest free match to Mila’s TT Norms Pro) for the title, headings, caption labels and running head, Latin Modern for body and maths, section numbers and caption labels in Mila purple #662E7D, citations in UdeM blue #0057AC, a running head with the short title and both brand marks, takeaway and prompt boxes, and an appendix that opens with its own contents list. The Meta-style commands are kept, so the Mem- π source compiles unchanged apart from its \documentclass line.

Vote with one letter. Pick E, F or G for the default title page, or name what you like in A to D and it will be folded in.



A Seed

B Meta FAIR

C ServiceNow

D LeapLab

E Ours, card

F Ours, band

G Ours, rule

Reinforcing Chain-of-Thought Reasoning with Self-Evolving Rubrics

Leheng Sheng^{1,2,*}, Wenchang Ma^{1,*}, Ruixin Hong¹,
Xiang Wang³, An Zhang^{3,†}, Tat-Seng Chua²

¹Seed, ByteDance, ²National University of Singapore,

³University of Science and Technology of China

*Equal Contribution, †Work done at ByteDance Seed, ‡Corresponding authors

Abstract

Despite chain-of-thought (CoT) playing crucial roles in LLM reasoning, directly rewarding it is difficult: training a reward model demands heavy human labeling efforts, and static RMs struggle with evolving CoT distributions and reward hacking. These challenges motivate us to seek an autonomous CoT rewarding approach that requires no human annotation efforts and can evolve gradually. Inspired by recent self-evolving training methods, we propose **RLCER** (Reinforcement Learning with CoT Supervision via Self-Evolving Rubrics), which enhances the outcome-centric RLVR by rewarding CoTs with self-proposed and self-evolving rubrics. We show that self-proposed and self-evolving rubrics provide reliable CoT supervision signals even without outcome rewards, enabling RLCER to outperform outcome-centric RLVR. Moreover, when used as in-prompt hints, these self-proposed rubrics further improve inference-time performance.

Date: September 27, 2026

Correspondence: An Zhang at an_zhang@ustc.edu.cn Wenchang Ma at mawenchang.99@bytedance.com

Project Page: <https://alphalab-ustc.github.io/rlcer-alphalab/>

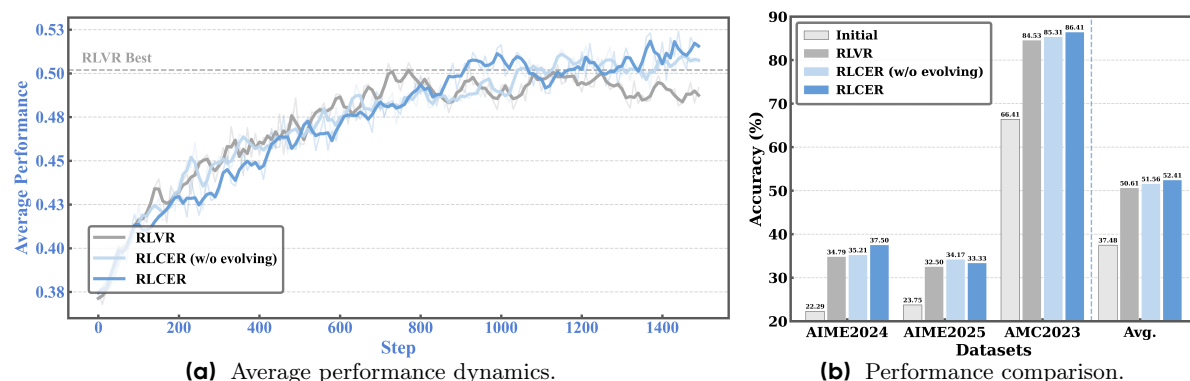


Figure 1 Performance across three math datasets on the 7B model. Training with RLCER leads to a higher performance ceiling, and the self-evolving rubrics further enhance reasoning performance.

1 Introduction

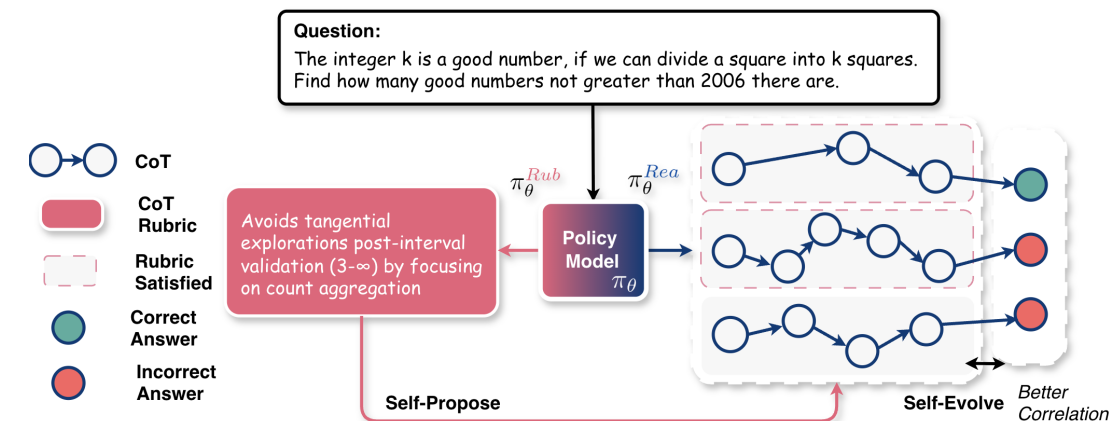


Figure 2 Key idea of reinforcement learning with CoT supervision via self-evolving rubrics (RLCER). The policy model π_θ acts as both the reasoner and the rubrics generator, self-generating and self-evolving the rubrics for CoT supervision, where the evolving direction is shaped towards the correlation with the rubrics satisfaction and the final answer correctness.

Chain-of-thought (CoT) [27] reasoning is essential for large language models (LLMs) to solve complex tasks, where its quality strongly affects the final-answer correctness [3, 9, 15, 39]. However, CoT is rarely rewarded or optimized explicitly, even often regarded as a by-product of outcome-centric optimization objectives in large-scale reinforcement learning with verifiable rewards (RLVR) [7, 13, 23]. This yields an underconstrained learning signal: for the same final answer, many distinct reasoning trajectories receive identical rewards, allowing optimization to drift toward shortcut or brittle strategies [33, 40]. Consequently, the model can converge to sub-optimal reasoning behaviors, limiting robustness and overall capability [16].

However, while important, directly rewarding or supervising CoTs remains challenging in practice, mainly for two reasons: (i) training additional reward models (RMs) for CoT supervision typically requires labor-intensive and fine-grained annotations [6, 16], and (ii) the policy’s CoT distribution shifts during training, yielding non-stationary and potentially biased supervision. **Can the policy model self-propose rubrics as CoT supervision criteria and self-evolve them during training with no human annotations?** If so, this could enable a new paradigm of self-improving reasoning, shifting RL from optimizing what LLMs answer to autonomously improving how LLMs think, potentially yielding “free-lunch” reasoning gains.

We find possibilities of achieving this from the recent advances in the research line of self-evolving training methods [10, 24], where a policy model π_θ can improve itself with little to no human intervention during training. On the one hand, π_θ itself can self-generate training data, such as verifiable question-answer pairs for RL or high-quality CoT trajectories for cold-starting reasoning [17, 19, 32, 37]. On the other hand, π_θ itself can self-assign reward signals by acting as a judge to evaluate its own solutions [4, 8, 29, 31]. A key idea shared by these methods is to let a single policy model π_θ play multiple roles under different prompts, serving not only as a reasoner that produces solutions but also as a data generator or a reward provider [31, 37], with all roles optimized jointly (e.g., via techniques such as multi-agent reinforcement learning [37]). Together, these capabilities make it plausible to self-propose supervision criteria (e.g., rubrics) for assessing CoT quality and to self-evolve as training progresses. However, how to adaptively propose and evolve such criteria in the self-evolving training paradigm remains largely underexplored.

To fill this gap, we propose **RLCER** (Reinforcement Learning with CoT Supervision via Self-Evolving Rubrics), empowering RLVR with self-proposed and self-evolving rubrics to supervise CoT reasoning. The core idea is to let the policy model π_θ self-propose CoT supervision criteria as natural-language rubrics (i.e., describing desirable reasoning properties like “avoid tangential explorations”) [11, 35], rewarding CoTs by their rubric satisfaction, and simultaneously improving the model’s rubric-generation ability throughout training. An overview of the key process is illustrated in Figure 2. Specifically, a single policy model π_θ is instantiated

Training Large Language Models to Reason in a Continuous Latent Space

Shibo Hao^{1,2,*}, Sainbayar Sukhbaatar¹, DiJia Su¹, Xian Li¹, Zhiting Hu², Jason Weston¹, Yuandong Tian¹

¹FAIR at Meta, ²UC San Diego

*Work done at Meta

Large language models (LLMs) are restricted to reason in the “language space”, where they typically express the reasoning process with a chain-of-thought (CoT) to solve a complex reasoning problem. However, we argue that language space may not always be optimal for reasoning. For example, most word tokens primarily ensure textual coherence and are not essential for reasoning, while some critical tokens require complex planning and pose huge challenges to LLMs. To explore the potential of LLM reasoning in an unrestricted latent space instead of using natural language, we introduce a new paradigm COCONUT (Chain of Continuous Thought). We utilize the last hidden state of the LLM as a representation of the reasoning state (termed “continuous thought”). Rather than decoding this into a word token, we feed it back to the LLM as the subsequent input embedding directly in the continuous space. This latent reasoning paradigm leads to the emergence of an advanced reasoning pattern: the continuous thought can encode multiple alternative next reasoning steps, allowing the model to perform a breadth-first search (BFS) to solve the problem, rather than prematurely committing to a single deterministic path like CoT. COCONUT outperforms CoT on certain logical reasoning tasks that require substantial search during planning, and shows a better trade-off between accuracy and efficiency.

Last updated: September 27, 2026

Code: <https://github.com/facebookresearch/coconut>



1 Introduction

Large language models (LLMs) have demonstrated remarkable reasoning abilities, emerging from extensive pretraining on human languages (Dubey et al., 2024; Achiam et al., 2023). While next token prediction is an effective training objective, it imposes a fundamental constraint on the LLM as a reasoning machine: the explicit reasoning process of LLMs must be generated in word tokens. For example, a prevalent approach, known as chain-of-thought (CoT) reasoning (Wei et al., 2022), involves prompting or training LLMs to generate solutions step-by-step using natural language. However, this is in stark contrast to certain human cognition results. Neuroimaging studies have consistently shown that the language network – a set of brain regions responsible for language comprehension and production – remains largely inactive during various reasoning tasks (Amalric and Dehaene, 2019; Monti et al., 2012, 2007, 2009; Fedorenko et al., 2011). Further evidence indicates that human language is optimized for communication rather than reasoning (Fedorenko et al., 2024).

A significant issue arises when LLMs use language for reasoning: the amount of reasoning required for each particular token varies greatly, yet current LLM architectures allocate nearly the same computing budget for predicting every token. Most tokens in a reasoning chain are generated solely for fluency, contributing little to the actual reasoning process. By contrast, some critical tokens require complex planning and pose huge challenges to LLMs. While previous work has attempted to fix these problems by prompting LLMs to generate succinct reasoning chains (Madaan and Yazdanbakhsh, 2022), or performing additional reasoning before generating some critical tokens (Zelikman et al., 2024), these solutions remain constrained within the language space and do not solve the fundamental problems. On the contrary, it would be ideal for LLMs to have the freedom to reason without any language constraints, and then translate their findings into language only when necessary.

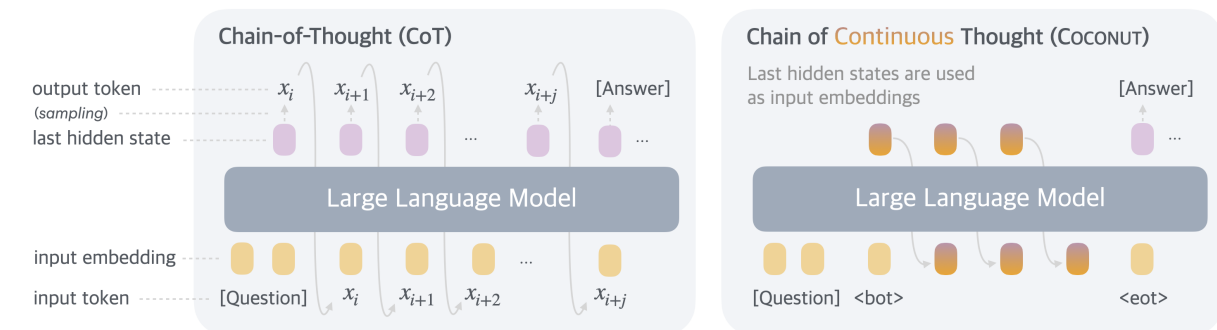


Figure 1 A comparison of Chain of Continuous Thought (COCONUT) with Chain-of-Thought (CoT). In CoT, the model generates the reasoning process as a word token sequence (e.g., $[x_i, x_{i+1}, \dots, x_{i+j}]$ in the figure). COCONUT regards the last hidden state as a representation of the reasoning state (termed “continuous thought”), and directly uses it as the next input embedding. This allows the LLM to reason in an unrestricted latent space instead of a language space.

In this work we instead explore LLM reasoning in a latent space by introducing a novel paradigm, COCONUT (Chain of Continuous Thought). It involves a simple modification to the traditional CoT process: instead of mapping between hidden states and language tokens using the language model head and embedding layer, COCONUT directly feeds the last hidden state (a continuous thought) as the input embedding for the next token (Figure 1). This modification frees the reasoning from being within the language space, and the system can be optimized end-to-end by gradient descent, as continuous thoughts are fully differentiable. To enhance the training of latent reasoning, we employ a multi-stage training strategy inspired by Deng et al. (2024), which effectively utilizes language reasoning chains to guide the training process.

Interestingly, our proposed paradigm leads to an efficient reasoning pattern. Unlike language-based reasoning, continuous thoughts in COCONUT can encode multiple potential next steps simultaneously, allowing for a reasoning process akin to breadth-first search (BFS). While the model may not initially make the correct decision, it can maintain many possible options within the continuous thoughts and progressively eliminate incorrect paths through reasoning, guided by some implicit value functions. This advanced reasoning mechanism surpasses traditional CoT, even though the model is not explicitly trained or instructed to operate in this manner, as seen in previous works (Yao et al., 2023; Hao et al., 2023).

Experimentally, COCONUT successfully enhances the reasoning capabilities of LLMs. For math reasoning (GSM8k, Cobbe et al., 2021), using continuous thoughts is shown to be beneficial to reasoning accuracy, mirroring the effects of language reasoning chains. This indicates the potential to scale and solve increasingly challenging problems by chaining more continuous thoughts. On logical reasoning including ProntoQA (Saparov and He, 2022), and our newly proposed ProsQA (Section 4) which requires stronger planning ability, COCONUT and some of its variants even surpasses language-based CoT methods, while generating significantly fewer tokens during inference. We believe that these findings underscore the potential of latent reasoning and could provide valuable insights for future research.

2 Related Work

Chain-of-thought (CoT) reasoning. We use the term chain-of-thought broadly to refer to methods that generate an intermediate reasoning process in language before outputting the final answer. This includes prompting LLMs (Wei et al., 2022; Khot et al., 2022; Zhou et al., 2022), or training LLMs to generate reasoning chains, either with supervised finetuning (Yue et al., 2023; Yu et al., 2023) or reinforcement learning (Wang et al., 2024; Havrilla et al., 2024; Shao et al., 2024; Yu et al., 2024a). Madaan and Yazdanbakhsh (2022) classified the tokens in CoT into symbols, patterns, and text, and proposed to guide the LLM to generate concise CoT based on analysis of their roles. Recent theoretical analyses have demonstrated the usefulness of CoT from the perspective of model expressivity (Feng et al., 2023; Merrill and Sabharwal, 2023; Li et al., 2024). By employing CoT, the effective depth of the transformer increases because the generated outputs are looped back to the input (Feng et al., 2023). These analyses, combined with the established effectiveness of CoT,

Mem- π : Adaptive Memory through Learning When and What to Generate

Xiaoqiang Wang^{1,2,3}, Chao Wang¹, Hadi Nekoei^{1,2,3}, Christopher Pal^{1,2,4,6}, Alexandre Lacoste¹, Spandana Gella^{1,5}, Bang Liu^{2,3,6}, Perouz Taslakian^{1,2,5}

¹ServiceNow AI Research, ²Mila – Quebec AI Institute, ³Université de Montréal, ⁴Polytechnique Montréal, ⁵McGill University, ⁶CIFAR AI Chair

We present Mem- π , a framework for adaptive memory in large language model (LLM) agents, where useful guidance is *generated on demand* rather than retrieved from external memory stores. Existing memory-augmented agents typically rely on similarity-based retrieval from episodic memory banks or skill libraries, returning static entries that often misalign with the current query context. In contrast, we model memory as a generative policy realized by a dedicated language or vision-language model with its own parameters, separate from the downstream agent, and fine-tuned specifically to produce context-specific guidance that cues the agent on how to perform complex tasks. The memory policy jointly decides *when* to produce guidance and *what* guidance to produce, trained with a decision-content decoupled reinforcement learning (RL) objective so that it abstains when generation would not help and otherwise produces concise, task-relevant guidance. Across diverse agentic benchmarks spanning web navigation, terminal tool use, and embodied environments, Mem- π consistently outperforms retrieval-based and RL-optimized memory baselines, achieving over 20% relative improvement on average.

Correspondence: bang.liu@umontreal.ca, perouz.taslakian@servicenow.com



1 Introduction

Large language models (LLMs) (Ouyang et al., 2022; Team et al., 2023; Hurst et al., 2024; DeepSeek-AI, 2026) have demonstrated remarkable capabilities on reasoning-intensive benchmarks (Liang et al., 2022; Srivastava et al., 2023; Phan et al., 2025; Deng et al., 2025) and shown potential as autonomous agents (Liu et al., 2025a) operating in real-world environments, enabling applications such as computer-use agents (Wang & Liu, 2025; Qin et al., 2025; Zhang et al., 2025a), deep research assistants (OpenAI, 2025; Li et al., 2025; Han et al., 2025), and automated scientific discovery systems (Lu et al., 2024; Schmidgall et al., 2025; Liu et al., 2026). Despite these advances, current LLMs remain limited by their stateless nature and cannot accumulate reusable experience across interactions (Sumers et al., 2023; Tao et al., 2024). To address this limitation, recent agent systems augment LLMs with external memory modules (Zhang et al., 2025d; Huang et al., 2026; Zhou et al., 2026), such as episodic memory banks (Zhong et al., 2024; Cai et al., 2025) and reusable skill libraries (Wang et al., 2023; Xia et al., 2026; Shi et al., 2026) distilled from prior interactions (Figure 1).

Existing memory-augmented agents collect memory fragments into a bank and retrieve relevant entries at inference time. Early approaches use *workflow-based memory* (Packer et al., 2023; Fu et al., 2024; Ouyang et al., 2025), where predefined rules govern memory construction, retrieval, and update (Zhao et al., 2024; Wang et al., 2025c). Recent work explores *learning-based memory* (Yan et al., 2025; Zhou et al., 2025; Zhang et al., 2025c, 2026b), optimizing memory operations end-to-end via downstream task outcomes. However, both lines remain constrained by a retrieval-based paradigm that reuses explicitly stored experiences. Retrieved memories are inherently static and often contain irrelevant (Wang et al., 2024b; Xu et al., 2026), partially aligned, or overly specific information (Yang et al., 2026a) that cannot adapt to the agent’s current context.

Cognitive science suggests a different view: human remembering is not a literal replay mechanism (Nosofsky et al., 1994; Ashby & Maddox, 2011) but a *constructive* process, where recollection is dynamically reconstructed from prior knowledge and the current context (Bartlett, 1932; Schacter et al., 1998; Schacter & Addis, 2007). Concurrent work such as ParaMem (Yao et al., 2026) and SEAM (Li et al., 2026) replaces retrieved memory

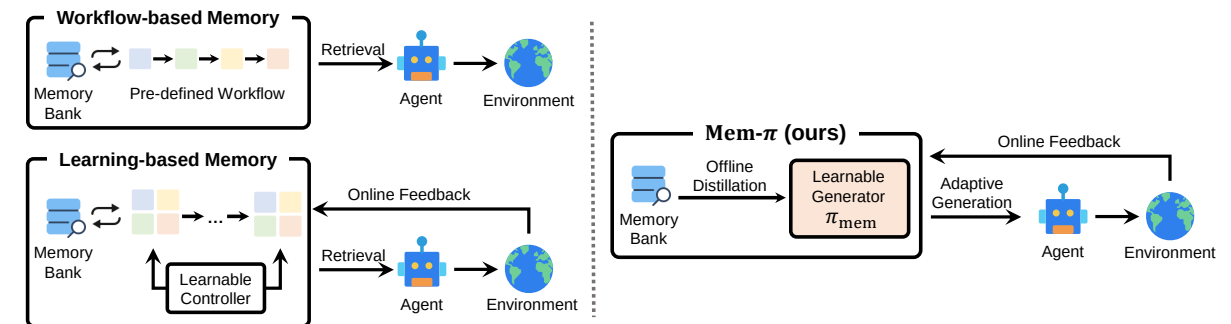


Figure 1 Comparison of (a) workflow-based memory systems, where memory operations are governed by predefined retrieval and update pipelines, (b) learning-based memory systems, where memory operations are jointly optimized with downstream agent outcomes, and (c) our Mem- π , which models memory as a generative policy π_{mem} separate from the downstream agent and internalizes reusable experience through offline experience distillation and online adaptation distillation.

with generated memory (Wang et al., 2025a; Wu et al., 2025b; Zhang et al., 2025b), but either applies it without conditioning on the current context or invokes generation as an always-on auxiliary step. This raises a reliability concern that is especially acute in agent settings, where memory is not the final task output but an intervention on a downstream agent. Under ambiguous, weakly grounded, or out-of-distribution contexts, generated guidance can be uninformative or even harmful, propagating hallucinated cues into agent actions.

Building on this, we present Mem- π , a framework for adaptive memory generation in LLM agents. Rather than retrieving fixed entries or unconditionally generating auxiliary guidance, Mem- π models memory as a parametric policy π_{mem} that learns both *when* to generate and *what* to generate. Conditioned on the agent context (*i.e.*, task instructions and environment observations), Mem- π produces concise, task-adaptive guidance from reusable experience encoded in its parameters.

Encoding experience in an Mem- π model’s parameters brings several advantages. First, its memory footprint is bounded by model size rather than the number of accumulated experiences, reducing the growing memory-management overhead associated with merging (Yin et al., 2024; Hu et al., 2024) and forgetting (Zhong et al., 2024). Second, since π_{mem} synthesizes guidance on demand rather than copying stored entries, it can fuse signals from many past experiences into a single context-specific hint, unlike top- k retrieval, which may split them across fragments or omit them beyond the cutoff (Jiang et al., 2023; Asai et al., 2023; Jeong et al., 2024). Finally, this framework separates specialization from execution: a smaller private local model can be fine-tuned as π_{mem} and plugged into a larger or frontier agent model to leverage broader reasoning capabilities.

We train Mem- π in two stages. *Experience distillation* first compresses an offline experience bank into the memory policy via supervised learning, internalizing reusable behaviors. *Adaptation distillation* then refines the policy through reinforcement learning, using downstream task outcomes as the reward signal to align memory generation with task success. To ensure reliability, we incorporate *abstention* into π_{mem} , allowing it to skip memory generation when generation is unnecessary or uncertain. Specifically, we introduce a decision-content decoupled objective built on Group Relative Policy Optimization (GRPO) (Shao et al., 2024) that separates *when* to generate from *what* to generate. The objective uses structured counterfactual rollouts to compare the two branches, decomposing learning into decision-level and content-level advantages and enabling adaptive memory behavior: the policy generates guidance only when it improves downstream task outcomes, and abstains otherwise.

We evaluate Mem- π across diverse agent benchmarks spanning web navigation (WEBARENA (Zhou et al., 2023), WORKARENA (Drouin et al., 2024)), terminal tool use (LIFELONGAGENTBENCH (Zheng et al., 2025)), and text-based embodied environments (ALFWORLD (Shridhar et al., 2020b)). Adaptive memory generation consistently outperforms retrieval-based memory baselines across all four benchmarks, yielding a 20% relative gain over the base agent on average, with the relative gain on WEBARENA approaching 50%.



September 27, 2026

Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?

Yang Yue^{1*†}, Zhiqi Chen^{1*}, Rui Lu¹, Andrew Zhao¹, Zhaokai Wang², Yang Yue¹, Shiji Song¹, and Gao Huang^{1✉}

¹LeapLab, Tsinghua University ²Shanghai Jiao Tong University

* Equal Contribution † Project Lead ✉ Corresponding Author

Reinforcement Learning with Verifiable Rewards (RLVR) has recently demonstrated notable success in enhancing the reasoning performance of large language models (LLMs), particularly in mathematics and programming tasks. It is widely believed that, similar to how traditional RL helps agents to explore and learn new strategies, RLVR enables LLMs to continuously self-improve, thus acquiring novel reasoning abilities that exceed the capacity of the corresponding base models. In this study, we take a critical look at the *current state of RLVR* by systematically probing the reasoning capability boundaries of RLVR-trained LLMs across various model families, RL algorithms, and math/coding/visual reasoning benchmarks, using pass@ k at large k values as the evaluation metric. While RLVR improves sampling efficiency towards correct paths, we surprisingly find that current training *rarely* elicit fundamentally new reasoning patterns. We observe that while RLVR-trained models outperform their base models at smaller values of k (e.g., $k=1$), base models achieve higher pass@ k score when k is large. Moreover, we observe that the reasoning capability boundary of LLMs often narrows as RLVR training progresses. Further coverage and perplexity analysis shows that the reasoning paths generated by RLVR models are already included in the base models' sampling distribution, suggesting that their reasoning abilities originate from and are *bounded* by the base model. From this perspective, treating the base model as an upper bound, our quantitative analysis shows that six popular RLVR algorithms perform similarly and remain far from optimal in fully leveraging the potential of the base model. In contrast, we find that distillation can introduce new reasoning patterns from the teacher and genuinely expand the model's reasoning capabilities. Taken together, our findings suggest that current RLVR methods have not fully realized the potential of RL to elicit genuinely novel reasoning abilities in LLMs. This underscores the need for improved RL paradigms, such as effective exploration mechanism, more deliberate and large-scale data curation, fine-grained process signal, and multi-turn agent interaction, to unlock this potential.

Project Page: <https://limit-of-RLVR.github.io>

1. Introduction

The development of reasoning-centric large language models (LLMs), such as OpenAI-o1 (Jaech et al., 2024), DeepSeek-R1 (Guo et al., 2025), and Kimi-1.5 (Team et al., 2025), has significantly advanced the frontier of LLM capabilities, particularly in solving complex logical tasks involving mathematics and programming. In contrast to traditional instruction-tuned approaches that rely on human-curated

The first author Yang Yue (乐洋) and the sixth author Yang Yue (乐阳) share the same English name but different Chinese names. Correspond to: {le-y22, zq-chen23}@mails.tsinghua.edu.cn, gaohuang@tsinghua.edu.cn.

Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?

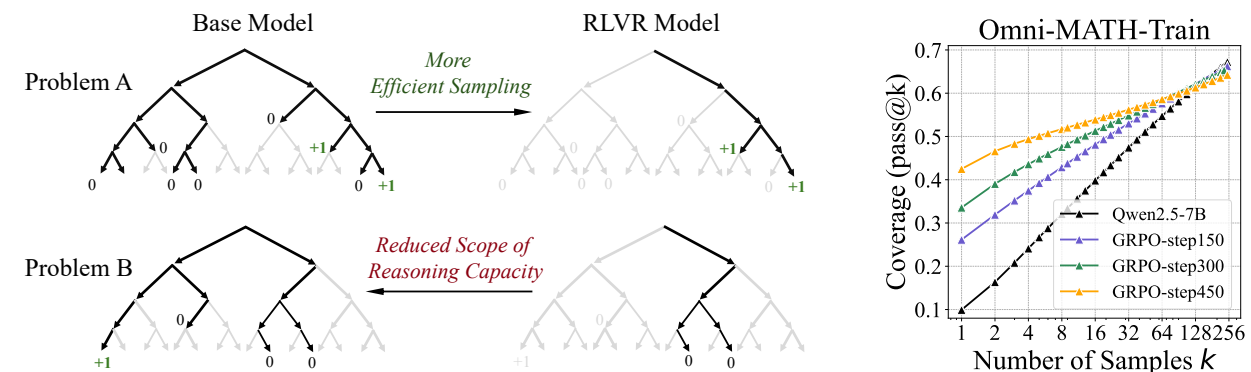


Figure 1: **(Left)** The effect of current RLVR on LLM's reasoning ability. Search trees are generated by repeated sampling from the base and RLVR-trained models for a given problem. Grey indicates paths that are unlikely to be sampled by the model, while black indicates paths that are likely to be sampled. Green indicates correct paths, which has positive rewards. Our key finding is that all reasoning paths in the RLVR model are already present in the base model. For certain problems like Problem A, RLVR training biases the distribution toward rewarded paths, improving sampling efficiency. However, this comes at the cost of reduced scope of reasoning capacity: For other problems like Problem B, the base model contains the correct path, whereas that of the RLVR model does not. **(Right)** As RLVR training progresses, the average performance (i.e., pass@1) improves, but the coverage of solvable problems (i.e., pass@256) decreases, indicating a reduction in LLM's reasoning boundary.

annotations (Achiam et al., 2023; Grattafiori et al., 2024), the key driver behind this leap forward is large-scale *Reinforcement Learning with Verifiable Rewards* (RLVR) (Lambert et al., 2024; Guo et al., 2025). RLVR starts with a pretrained base model or one fine-tuned on long chains of thought (CoT) data, optimizing it via reinforcement learning based on simple, automatically computable rewards. These rewards are determined by whether the model's output matches a ground-truth solution in mathematics or passes unit tests in code, thus enabling scalability without human labeling. This framework has gained significant attention due to its simplicity and practical effectiveness. In traditional RL settings such as game playing (e.g., Atari, Go), agents often autonomously discover new strategies and surpass even human-level performance through self-improvement (Mnih et al., 2015; Silver et al., 2017). Inspired by this success, it is widely believed that RLVR similarly enables LLMs to autonomously develop novel reasoning patterns, including enumeration, self-reflection, and iterative refinement, surpassing the capabilities of their base models (Guo et al., 2025). Consequently, RLVR has been considered a promising path toward continuously self-evolving LLMs, potentially bringing us closer to more powerful intelligence (Guo et al., 2025).

However, despite its empirical success, the underlying effectiveness of current RLVR remains underexamined. This raises a fundamental question: *Does current RLVR genuinely enable LLMs to acquire novel reasoning abilities—similar to how traditional RL discovers new strategies through exploration—or does it simply utilize reasoning patterns already in the base model?*

To rigorously answer this question, we must first assess the reasoning capability boundaries of both base and RLVR-trained models. Traditional evaluation metrics rely on average score from greedy decoding or nucleus sampling (Holtzman et al., 2020), which reflects average-case behavior. However, these metrics risk underestimating the true potential of a model, especially if it fails on difficult problems after limited attempts, despite being capable of solving them with more sampling. To overcome this limitation, we adopt the pass@ k metric (Brown et al., 2024), where a problem is considered solved if any of the k sampled outputs is correct. By allowing multiple attempts, pass@ k reveals whether a model has the potential to solve a problem. The average pass@ k across a dataset thus reflects the proportion of problems a model can potentially solve within k trials, offering a more robust view of its reasoning boundary. This provides a rigorous test on whether the RLVR training yields fundamentally transcending capacity, enabling the model to solve problems that the base model cannot.

Using the pass@ k metric, we conduct extensive experiments across various benchmarks, covering multiple LLM families, model sizes, and RLVR algorithms to compare base models with their RLVR-trained

Title of Your Paper in Two Lines at Most: A Subtitle That Names the Method

First Author^{1,2,*}, Second Author^{1,2,*}, Third Author^{1,3}, Senior Author^{1,2,†}

¹Mila – Quebec AI Institute ²Université de Montréal ³McGill University

*Equal contribution [†]Corresponding author

Abstract

This template typesets arXiv preprints for authors at Université de Montréal and Mila. It keeps what works in the industry preprint styles of Meta FAIR, ByteDance Seed, ServiceNow and LeapLab, a front matter that reads as one block, sans-serif headings over a serif body, and metadata a reader can click, and sets them in Mila purple and UdeM blue. A single class option chooses among three title pages. The author, affiliation and metadata commands follow the Meta FAIR convention, so a paper written for the Meta FAIR, ServiceNow or ByteDance Seed class moves here by changing its `\documentclass` line. Replace this paragraph with your abstract and keep it to one paragraph of about 150 to 250 words.

 **Date:** September 27, 2026

 **Correspondence:** `first.author@mila.quebec`, `senior.author@umontreal.ca`

 **Code:** `https://github.com/your-org/your-repo`

 **Hugging Face:** `https://huggingface.co/your-org`

1 Introduction

Industry labs publish their arXiv preprints in house styles that make a paper recognisable before its title is read. Meta FAIR sets the whole front matter in a tinted card ([Hao et al., 2024](#)), ByteDance Seed rules off a centred title under a logo header ([Sheng et al., 2026](#)), ServiceNow frames the card in its brand green and adds link lines for code and models ([Wang et al., 2026](#)), and LeapLab puts the abstract on a grey panel below a logo and date ([Yue et al., 2025](#)). This class brings those choices to Université de Montréal and Mila, with the official UdeM and Mila marks and colours.

The body text is Latin Modern, so mathematics such as the attention operator of [Vaswani et al. \(2017\)](#),

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (1)$$

matches the surrounding text. Everything that carries the brand, the title block, headings, caption labels and the running head, is set in Inter, the closest free match to Mila’s own typeface. [Figure 1](#) is where the overview figure of a paper would go, and [Section 2](#) explains the class options and the front matter commands.

2 Using the class

2.1 Class options

The class builds on `article`, so every `article` option such as `11pt` or `twocolumn` passes through unchanged. [Table 1](#) lists the two options the class adds. The title style is the only choice that changes how the paper looks at a glance. The `card` style follows Meta FAIR and ServiceNow, `band` puts a full-width purple band with white logos across the top of the first page, and `rule` follows ByteDance Seed and LeapLab with colour logos in the header and a ruled, centred title.

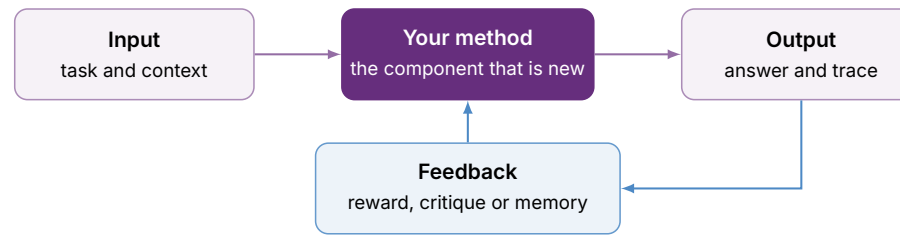


Figure 1 Overview figure placeholder. The method node is filled in Mila purple and the feedback path is drawn in UdeM blue, the two brand colours this class defines as `milapurple` and `udemblue`.

Table 1 Options added by `udemmila.cls`. Any other option is passed to `article`.

Option	Values	Effect
<code>titlestyle</code>	<code>card</code> (default), <code>band</code> , <code>rule</code>	Layout of the first page
<code>numbers</code>	<code>flag</code>	Numeric citations [1] instead of (Author, Year)

2.2 Front matter

The front matter is declared in the preamble and printed by `\maketitle`. [Table 2](#) lists the commands. Authors take their affiliation marks as an optional argument, and the abstract is a command rather than an environment, both as in the Meta FAIR class. Every metadata line is optional, and `\metadata` adds a line of any other kind with a Font Awesome icon.

3 Figures, tables and boxes

Captions open with a purple label in Inter. A caption that fits on one line is centred, and a longer one is justified. Table captions sit above the table. [Table 3](#) shows the recommended results layout, with `booktabs` rules, the best entry in bold, the runner-up underlined and the proposed method on a row tinted with `umtint`.

Two boxes cover the content arXiv papers in this area most often set apart. The `takeaway` box states a finding the reader should keep, and the `promptbox` holds a prompt or a model output in Inconsolata, with verbatim content allowed inside.

Takeaway

State the finding in one or two sentences with the number that supports it, for example that the method improves the average score by 3.4 points over the strongest published baseline in [Table 3](#).

Prompt template for the judge model

You are grading a response to the task below.
Task: {task}
Response: {response}
Reply with a score from 1 to 10 and one sentence of justification.

4 Conclusion

Replace this sample text with your paper. The appendix below starts with `\beginappendix`, which opens a new page, prints the appendix heading and lists the appendix sections, and `\beginappendix*` leaves the list out.

References

- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- Leheng Sheng, Wenchang Ma, Ruixin Hong, Xiang Wang, An Zhang, and Tat-Seng Chua. Reinforcing chain-of-thought reasoning with self-evolving rubrics. *arXiv preprint arXiv:2602.10885*, 2026.

Title of Your Paper in Two Lines at Most: A Subtitle That Names the Method

First Author^{1,2,*}, Second Author^{1,2,*}, Third Author^{1,3}, Senior Author^{1,2,+}

¹Mila – Quebec AI Institute ²Université de Montréal ³McGill University

*Equal contribution +Corresponding author

Abstract

This template typesets arXiv preprints for authors at Université de Montréal and Mila. It keeps what works in the industry preprint styles of Meta FAIR, ByteDance Seed, ServiceNow and LeapLab, a front matter that reads as one block, sans-serif headings over a serif body, and metadata a reader can click, and sets them in Mila purple and UdeM blue. A single class option chooses among three title pages. The author, affiliation and metadata commands follow the Meta FAIR convention, so a paper written for the Meta FAIR, ServiceNow or ByteDance Seed class moves here by changing its `\documentclass` line. Replace this paragraph with your abstract and keep it to one paragraph of about 150 to 250 words.

Date: September 27, 2026

Correspondence: `first.author@mila.quebec`, `senior.author@umontreal.ca`

Code: `https://github.com/your-org/your-repo`

Hugging Face: `https://huggingface.co/your-org`

1 Introduction

Industry labs publish their arXiv preprints in house styles that make a paper recognisable before its title is read. Meta FAIR sets the whole front matter in a tinted card (Hao et al., 2024), ByteDance Seed rules off a centred title under a logo header (Sheng et al., 2026), ServiceNow frames the card in its brand green and adds link lines for code and models (Wang et al., 2026), and LeapLab puts the abstract on a grey panel below a logo and date (Yue et al., 2025). This class brings those choices to Université de Montréal and Mila, with the official UdeM and Mila marks and colours.

The body text is Latin Modern, so mathematics such as the attention operator of Vaswani et al. (2017),

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (1)$$

matches the surrounding text. Everything that carries the brand, the title block, headings, caption labels and the running head, is set in Inter, the closest free match to Mila’s own typeface. Figure 1 is where the overview figure of a paper would go, and Section 2 explains the class options and the front matter commands.

2 Using the class

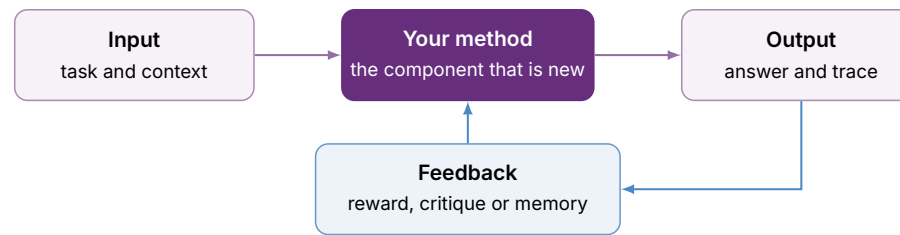


Figure 1 Overview figure placeholder. The method node is filled in Mila purple and the feedback path is drawn in UdeM blue, the two brand colours this class defines as `milapurple` and `udemblue`.

Table 1 Options added by `udemmila.cls`. Any other option is passed to `article`.

Option	Values	Effect
<code>titlestyle</code>	<code>card</code> (default), <code>band</code> , <code>rule</code>	Layout of the first page
<code>numbers</code>	<code>flag</code>	Numeric citations [1] instead of (Author, Year)

2.1 Class options

The class builds on `article`, so every `article` option such as `11pt` or `twocolumn` passes through unchanged. Table 1 lists the two options the class adds. The title style is the only choice that changes how the paper looks at a glance. The `card` style follows Meta FAIR and ServiceNow, `band` puts a full-width purple band with white logos across the top of the first page, and `rule` follows ByteDance Seed and LeapLab with colour logos in the header and a ruled, centred title.

2.2 Front matter

The front matter is declared in the preamble and printed by `\maketitle`. Table 2 lists the commands. Authors take their affiliation marks as an optional argument, and the abstract is a command rather than an environment, both as in the Meta FAIR class. Every metadata line is optional, and `\metadata` adds a line of any other kind with a Font Awesome icon.

3 Figures, tables and boxes

Captions open with a purple label in Inter. A caption that fits on one line is centred, and a longer one is justified. Table captions sit above the table. Table 3 shows the recommended results layout, with booktabs rules, the best entry in bold, the runner-up underlined and the proposed method on a row tinted with `umtint`.

Two boxes cover the content arXiv papers in this area most often set apart. The `takeaway` box states a finding the reader should keep, and the `promptbox` holds a prompt or a model output in Inconsolata, with verbatim content allowed inside.

Takeaway

State the finding in one or two sentences with the number that supports it, for example that the method improves the average score by 3.4 points over the strongest published baseline in Table 3.

Prompt template for the judge model

You are grading a response to the task below.
Task: {task}
Response: {response}
Reply with a score from 1 to 10 and one sentence of justification.

4 Conclusion

Replace this sample text with your paper. The appendix below starts with `\beginappendix`, which opens a new page, prints the appendix heading and lists the appendix sections, and `\beginappendix*` leaves the list out.